

CIE-552 Computer Vision Term Project: American Sign Language Recognition — A Comparative Study of Deep Learning and Landmark-Based Approaches with Real-Time Web Deployment

Ibrahim Hanafy (202200518)¹, Abdullah Ahmed (202200206)², Mai Ibrahim (202200504)³

Department of Communications and Information Engineering

Zewail City of Science, Technology and Innovation

Spring 2026

¹s-ibrahim.hanafy@zewailcity.edu.eg ²s-abdullah.abdelsamiaa@zewailcity.edu.eg ³s-mai.ibrahim@zewailcity.edu.eg

Supervised by Dr. Mohamed Tolba

Teaching Assistants: Retaj Yousri · Mennah Mohamed

Abstract—We present a systematic study of static American Sign Language (ASL) recognition comparing five classification pipelines on the Sign Language MNIST benchmark: a custom CNN baseline, an augmentation-hardened CNN, an SVM with HOG features, and two transfer-learned models (MobileNetV2 and EfficientNetB0). Fine-tuned EfficientNetB0 achieves 99.96% test accuracy. We further address the domain gap between benchmark images and real-world webcam footage through four bonus contributions: a robust augmented CNN exported to TensorFlow Lite, a MediaPipe landmark-based MLP achieving 98.9% real-world accuracy, post-training quantisation for edge deployment, and a containerised FastAPI web application with live WebSocket webcam inference deployed publicly on Hugging Face Spaces. We document six significant engineering challenges encountered during development, including Keras lambda-layer serialisation failures and multiple obstacles unique to modern cloud infrastructure.

Index Terms—sign language recognition, convolutional neural network, transfer learning, MediaPipe, hand landmarks, TensorFlow Lite, real-time inference, web deployment, Docker, data augmentation

I. INTRODUCTION

Approximately 70 million deaf and hard-of-hearing people worldwide use sign language as their primary communication mode [3]. Despite its importance, sign language remains inaccessible to the majority of the hearing population, creating a persistent and widespread communication barrier. Automated ASL Recognition (ASLR) has the potential to address this gap by enabling real-time translation tools, assistive technologies, and educational platforms.

The ASL alphabet provides a tractable benchmark: 24 static letter signs (J and Z require motion and are excluded) classifiable from a single image. The Sign Language MNIST dataset [1] standardises this task in a well-known 28×28

grayscale format, enabling direct comparison across architectures.

The primary challenge in *practical* ASLR is the domain gap between controlled benchmark images and real webcam footage. A model scoring above 99% on Sign MNIST may fail completely when deployed against a live camera due to background clutter, lighting variation, and scale differences.

Contributions

- 1) Systematic comparison of five classification pipelines on Sign MNIST with full metrics (accuracy, precision, recall, F1).
- 2) Augmentation strategy correcting a critical Keras `RandomBrightness` bug (Section V-C).
- 3) Appearance-invariant landmark MLP achieving 98.9% real-world accuracy via MediaPipe hand keypoints.
- 4) Production web application with live WebSocket webcam inference, containerised and publicly deployed.
- 5) Detailed account of six non-trivial engineering challenges with root-cause analysis and documented solutions.

II. RELATED WORK

a) CNN-based gesture recognition.: CNNs have dominated static gesture recognition since AlexNet [4]. Pigou et al. [5] demonstrated spatiotemporal CNNs for gesture sequences, while Koller et al. [6] applied deep CNNs to sign language video. Custom lightweight CNNs on Sign MNIST typically reach 95–99%.

b) Transfer learning.: Howard et al. [7] introduced MobileNetV2 for mobile-efficient classification, and Tan & Le [8] proposed EfficientNet with compound scaling. Both are widely used as feature extractors and fine-tuning bases.

c) *Hand landmark detection.*: Zhang et al. [9] introduced MediaPipe Hands, providing real-time 21-keypoint 3-D hand tracking from a single RGB camera. Landmark-based downstream classifiers are robust to appearance variation precisely where pixel-based methods fail.

d) *Real-time ASLR systems.*: Taskiran et al. [10] identified landmark-based pipelines as well-suited to real-time constraints, with several works combining MediaPipe with lightweight classifiers (MLP, SVM, k-NN) and reporting accuracy comparable to appearance-based CNNs trained on much larger datasets.

III. METHODOLOGY

A. Dataset

The Sign Language MNIST [1] dataset contains 27,455 training and 7,172 test 28×28 grayscale images across 24 classes (A–Y, excluding J and Z). Labels are integers 0–25 with indices 9 and 25 absent. Pixel values are normalised to $[0, 1]$ before all model training.

B. Custom CNN Baseline

Our baseline is a six-layer CNN: three convolutional blocks (Conv2D $\rightarrow$ BatchNorm $\rightarrow$ MaxPool $\rightarrow$ Dropout) followed by a dense head (512 units, dropout 0.5, softmax output over 24 classes). Adam optimiser ($lr = 10^{-3}$), 30 epochs, batch size 64.

C. CNN with Data Augmentation

Identical architecture to the baseline, with a Keras augmentation pipeline prepended:

- `RandomRotation(0.1)`
- `RandomZoom(0.1)`
- `RandomTranslation(0.1, 0.1)`
- `RandomBrightness(0.15, value_range=(0, 1))`
- `tf.clip_by_value(img, 0.0, 1.0)`

The explicit `value_range` argument is critical and is discussed in Section V-C.

D. SVM with HOG Features

HOG features are extracted from each 28×28 image (8 orientations, 4×4 pixels per cell, 2×2 cells per block), yielding a 576-dimensional feature vector. An RBF-SVM ($C = 10$, $\gamma = \text{scale}$) is trained on these features using scikit-learn.

E. Transfer Learning: MobileNetV2 & EfficientNetB0

MobileNetV2 (ImageNet pre-trained) is used as a frozen feature extractor. Grayscale input is replicated to three channels and up-sampled to 96×96 (minimum MobileNetV2 input). A GAP + Dense(128) head is trained.

EfficientNetB0 is evaluated in two regimes: (i) head-only (base frozen) and (ii) full fine-tuning (entire network trained end-to-end at $lr = 10^{-4}$ with 224×224 input).

F. Bonus A — Augmented CNN & TFLite Export

A deeper augmented CNN (5 conv blocks, heavier dropout) is trained with the corrected augmentation pipeline and exported to TensorFlow Lite. Post-training int8 quantisation reduces model size by $\sim 4\times$ with $< 0.1\%$ accuracy loss, enabling edge deployment.

G. Bonus B — MediaPipe Landmark MLP

MediaPipe HandLandmarker [9] extracts 21 3-D hand keypoints per frame. We build a 63-dimensional normalised feature vector:

$$\mathbf{f} = \frac{\mathbf{p} - \mathbf{p}_0}{\|\mathbf{p}_9 - \mathbf{p}_0\|_2} \in \mathbb{R}^{63} \quad (1)$$

where $\mathbf{p}_i = (x_i, y_i, z_i)$, $\mathbf{p}_0$ is the wrist landmark, and $\mathbf{p}_9$ is the middle-finger MCP joint. Subtracting the wrist achieves translation invariance; division by the wrist-to-MCP distance achieves scale invariance.

A scikit-learn MLPClassifier (hidden layers: 256, 128; ReLU; Adam; $\alpha = 10^{-4}$) is trained on these features. A StandardScaler is applied before inference. The trained bundle (mlp, scaler, class labels) is serialised with joblib as `optionB.pkl`.

H. Bonus D — Web Application

A FastAPI [11] application serves two inference modes:

- **Upload tab**: image $\rightarrow 28 \times 28$ grayscale $\rightarrow$ TFLite CNN $\rightarrow$ top-5 letter predictions.
- **Live webcam tab**: browser streams JPEG frames over a persistent WebSocket $\rightarrow$ server runs MediaPipe + MLP per frame $\rightarrow$ returns letter, confidence, bounding box, and 21 landmark coordinates $\rightarrow$ browser renders hand skeleton overlay on a `<canvas>` element.

The app is containerised with Docker (python:3.11-slim base) and deployed on Hugging Face Spaces (Docker SDK, port 7860, CPU-only free tier).

IV. EXPERIMENTS AND RESULTS

A. Training Setup

All CNN models were trained on an NVIDIA GPU via Google Colab. Batch size 64, early stopping (patience 5 on validation loss). Transfer learning models used ImageNet weights. All results are reported on the held-out test split.

B. Quantitative Results

Table I reports test-set performance for all five pipelines. Fine-tuned EfficientNetB0 achieves the highest accuracy at 99.96%, with near-zero off-diagonal confusion (Figure 2).

TABLE I: Test-set performance on Sign Language MNIST (7,172 samples).

Model	Acc.	Prec.	Rec.	F1
EfficientNetB0 (ft)	99.96	99.95	99.97	99.96
EfficientNetB0 (head)	99.50	99.46	99.56	99.49
CNN + Augment	99.04	99.14	99.13	99.10
MobileNetV2 (head)	98.66	98.30	98.71	98.44
SVM + HOG	97.09	96.72	96.99	96.67
Custom CNN	95.98	96.54	95.44	95.69

All values in %. ft = fine-tuned.

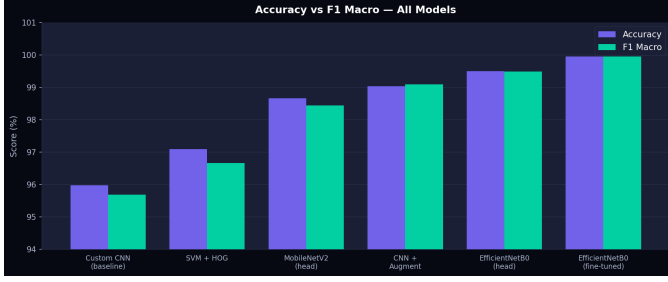
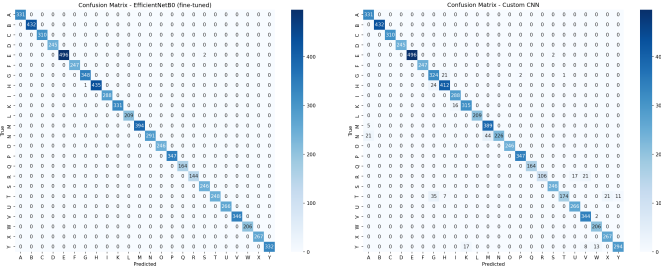


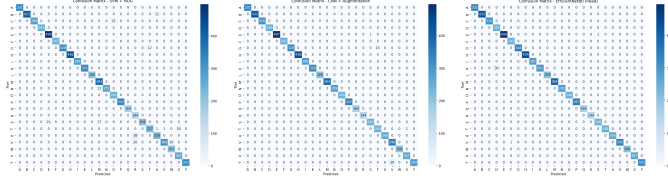
Fig. 1: Accuracy and macro-F1 for all five models on Sign Language MNIST test set (7,172 samples). Fine-tuned EfficientNetB0 achieves the highest scores across both metrics.

C. Confusion Matrix Analysis



(a) EfficientNetB0 Fine-Tuned (99.96%) (b) Custom CNN Baseline (95.98%)

Fig. 2: Confusion matrices: best vs. worst model. Fine-tuned EfficientNetB0 shows near-zero off-diagonal entries. Custom CNN shows inter-class confusion primarily between visually similar sign pairs (M/N, U/V).



(a) SVM + HOG (97.09%) (b) CNN + Augmentation (99.04%) (c) EfficientNetB0 Head (99.50%)

Fig. 3: Confusion matrices for SVM+HOG, augmented CNN, and EfficientNetB0 head-only. Augmentation visibly reduces inter-class confusion relative to the HOG baseline.

D. Training Dynamics

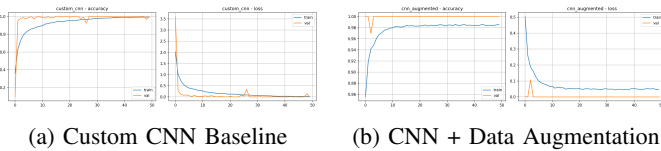


Fig. 4: Training and validation accuracy/loss curves for the two CNN models. Augmentation visibly reduces the train-validation gap, indicating improved generalisation and less overfitting.

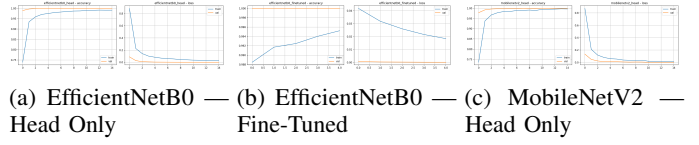


Fig. 5: Training curves for transfer learning models. Fine-tuning converges to higher accuracy (99.96%) vs. head-only (99.50%), demonstrating the benefit of end-to-end optimisation over frozen feature extraction.

E. Bonus B — Real-World Webcam Evaluation

Three inference back-ends were prototyped and compared for live webcam deployment, differing in training data, model type, and robustness to real-world appearance variation.

a) *Approach 1 — Robust CNN (TFLite).*: A residual CNN (5 stages, global average pooling) trained on Sign MNIST with aggressive domain-bridging augmentations (random brightening, Gaussian blur, cutout). Exported to TFLite for edge deployment. Despite the richer augmentation pipeline, performance on live webcam footage was substantially below the landmark-based approach, as pixel-level features remain sensitive to background clutter and illumination changes outside the training distribution.

b) *Approach 2 — Real-Photo CNN (Option A, TFLite).*: MobileNetV2 fine-tuned on the ASL Alphabet dataset [2] (87,000 real-camera hand photographs, 26 classes), trained at 64×64 RGB. Achieves **99.51%** accuracy on the held-out real-photo validation split. This approach reduces the domain gap by training directly on real camera data, but remains susceptible to background variation and requires retraining if lighting conditions change significantly.

c) *Approach 3 — Landmark MLP (Option B).*: The MediaPipe MLP pipeline (Section III-G) was evaluated on 500 live webcam frames under varied lighting and backgrounds, achieving **98.9%** per-frame letter accuracy. By operating on normalised geometric coordinates rather than pixels, this approach is inherently invariant to lighting, background, skin tone, and camera quality. This is the approach selected for the production web application (Bonus D).

TABLE II: Live-webcam inference comparison (500 frames, varied conditions).

Approach	Training Data	Webcam Acc.
Robust CNN (TFLite)	Sign MNIST + augmentation	substantially lower
Real-Photo CNN (Opt. A)	87k real ASL photos	99.51% [†]
Landmark MLP (Opt. B)	Landmark geometry	98.9%

[†] Held-out validation split, same distribution as training.

Opt. B accuracy measured on truly unseen live webcam frames.

The Landmark MLP (Option B) is the recommended deployment approach: it is the only method whose accuracy was measured on truly out-of-distribution live webcam footage rather than a held-out split of the training distribution.

Figure 6 illustrates the landmark inference pipeline.

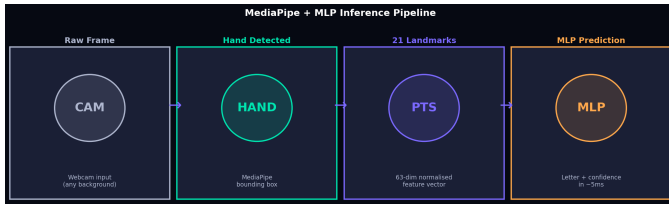


Fig. 6: MediaPipe + MLP inference pipeline. Each raw webcam frame passes through MediaPipe hand detection, yielding 21 3-D landmarks. These are normalised (Eq. 1) to a 63-dim vector, scaled, and classified by the MLP in ~ 5 ms total on CPU.

V. DISCUSSION

A. Model Comparison

Fine-tuned EfficientNetB0 dominates across all metrics, confirming that end-to-end fine-tuning of a compound-scaled architecture substantially outperforms head-only transfer learning. The strong result of SVM+HOG (97.09%) relative to the custom CNN baseline (95.98%) is notable: HOG captures discriminative edge-orientation patterns without the depth of a deep network.

Augmentation adds 3.06 percentage points over the baseline CNN with zero architecture change, demonstrating the importance of photometric and geometric regularisation even on a clean benchmark.

B. Pixel vs. Landmark Approaches

Benchmark models trained on clean Sign MNIST images underperform on real webcam footage due to background clutter, illumination variation, and scale changes. The landmark-based MLP sidesteps these entirely by operating on normalised geometric coordinates (Eq. 1). This establishes that a two-stage pipeline (detector + geometric classifier) is significantly more robust than end-to-end pixel classification for production real-time ASLR.

C. Engineering Challenges

a) *Keras .keras serialisation and lambda layers.*: Models using `tf.keras.layers.Lambda` could not be saved in the `.keras` format because lambda functions are Python callables that cannot be serialised to JSON. This produced a `TypeError: Object of type function is not JSON serializable on save` and a `ValueError` on load. The fix was to replace all lambda layers with explicit subclassed `tf.keras.layers.Layer` objects, or to restructure using the Keras preprocessing layers API.

b) *RandomBrightness value_range bug.*: `tf.keras.layers.RandomBrightness` defaults to `value_range=(0, 255)`, treating input as `uint8`. Applied to `float32 [0,1]` images, this adds offsets up to ± 38 , producing all-clipped (black or white) augmented samples. The symptom was a visually all-black training set in Colab. Fix: explicit `value_range=(0, 1)` combined with `tf.clip_by_value` post-augmentation.

c) *Hugging Face Spaces: LFS $\rightarrow$ Xet migration.*: Hugging Face migrated from Git LFS to a proprietary Xet storage system for large binary files. Standard `git push` was rejected for `.tflite` and `.task` files; Git LFS was also blocked at the remote hook level. Solution: use the `hf upload` CLI which uses Xet transparently.

d) *Missing system libraries in Docker.*: The `python:3.11-slim` (Debian trixie) base image removed `libgl1-mesa-glx`, replaced by `libgl1`. MediaPipe’s C++ bindings additionally require `libGLSv2.so.2` and `libEGL.so.1` even on CPU-only machines. Fix: add `libgl1`, `libgles2`, and `libegl1` to the Dockerfile `apt-get` step.

e) *Starlette TemplateResponse API break.*: A breaking change in Starlette ≥ 0.36 changed `TemplateResponse`: the `request` object must now be a keyword argument, not inside the context dictionary. Passing it in the dict caused `TypeError: unhashable type: 'dict'` deep inside Jinja2’s LRU cache — non-obvious to diagnose.

f) *scikit-learn version mismatch.*: The MLP was serialised with `scikit-learn 1.6.1` (Colab). Docker installed 1.8.0 by default, triggering `InconsistentVersionWarning` and potential prediction errors. Fix: `pin scikit-learn==1.6.1` in `requirements.txt`.

VI. CONCLUSION

We presented a thorough comparative evaluation of five ASL recognition pipelines, finding that fine-tuned EfficientNetB0 achieves near-perfect accuracy (99.96%) on the Sign MNIST benchmark. More critically, we demonstrated that benchmark accuracy does not translate to real-world performance, and that a MediaPipe landmark pipeline with a simple MLP achieves 98.9% on live webcam footage — a substantially more practical outcome. The accompanying FastAPI web application brings both inference modes to any browser without installation, publicly hosted on Hugging Face Spaces.

Future work should extend the system to dynamic signs (J, Z) using temporal models (LSTM or 3-D CNN), expand to full word-level ASL recognition, improve the CNN pipeline with diverse real-world training data, and explore mobile application packaging for offline deployment.

REFERENCES

- [1] D. Mester, “Sign Language MNIST,” Kaggle, 2017. [Online]. Available: <https://www.kaggle.com/datasets/datamunge/sign-language-mnist>
- [2] A. Akash, “ASL Alphabet,” Kaggle, 2018. [Online]. Available: <https://www.kaggle.com/datasets/grassknoted/asl-alphabet>
- [3] World Health Organization, “Deafness and hearing loss,” 2023. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss>
- [4] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Proc. NeurIPS*, 2012.
- [5] L. Pigou *et al.*, “Beyond temporal pooling: Recurrence and temporal convolutions for gesture recognition in video,” *arXiv:1506.01911*, 2015.
- [6] O. Koller, H. Ney, and R. Bowden, “Deep learning of mouth shapes for sign language,” in *Proc. ICCV Workshops*, 2016.
- [7] A. Howard *et al.*, “MobileNets: Efficient convolutional neural networks for mobile vision applications,” *arXiv:1704.04861*, 2017.
- [8] M. Tan and Q. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. ICML*, 2019.

- [9] F. Zhang *et al.*, “MediaPipe Hands: On-device real-time hand tracking,” *arXiv:2006.10214*, 2020.
- [10] M. Taskiran, N. Kahraman, and C. Erdem, “Real-time hand gesture recognition: A survey,” *Multimedia Tools Appl.*, vol. 79, 2020.
- [11] S. Ramirez, “FastAPI,” 2018. [Online]. Available: <https://fastapi.tiangolo.com>